# SMS Spam Detector

Complete Model Architecture & Design Decisions

This document is a comprehensive technical reference for every decision made during the design and training of the SMS Spam Detector — from raw data ingestion to the final deployed ensemble model.

## 1. Problem Statement

SMS spam is a global nuisance ranging from annoying promotional messages to active phishing attempts. The goal of this project is to build a binary text classifier that can distinguish between **spam** and **ham** (legitimate) SMS messages in real time. The key constraint is **high precision** — a false positive (marking a real message as spam) is worse than a false negative (missing spam). This shaped every modelling decision.

## 2. Dataset

| | |
|---|---|
| Source | SMS Spam Collection Dataset (UCI / Kaggle) |
| Total Messages | 5,572 SMS messages |
| Ham (Class 0) | ~4,825 messages (~87%) |
| Spam (Class 1) | ~747 messages (~13%) |
| Class Imbalance Ratio | ~6.5 : 1 (ham : spam) |
| Columns Used | label (target), message (raw SMS text) |
| Label Encoding | ham → 0,  spam → 1  (using sklearn LabelEncoder) |
| Duplicates Removed | Yes — duplicate rows dropped before modelling |
| Missing Values | None found |

The severe class imbalance (13% spam) made accuracy a misleading metric. Precision was chosen as the primary evaluation criterion throughout.

# 3. Exploratory Data Analysis (EDA)

Before any modelling, statistical properties of spam vs. ham messages were analysed to understand what features a classifier could exploit.

## 3.1 Message Length Statistics

| Metric | Ham (avg) | Spam (avg) |
|---|---|---|
| Character count | ~71 | ~138 |
| Word count | ~14 | ~27 |
| Sentence count | ~1.9 | ~2.1 |

Spam messages are on average nearly **twice as long** as ham messages. This was confirmed via histograms and pairplots.

## 3.2 Key Linguistic Patterns Observed

| Pattern | Spam | Ham |
|---|---|---|
| Urgency words ("NOW", "URGENT", "ACT") | Very frequent | Rare |
| Monetary symbols ($, £, %) | Frequent | Rare |
| Call-to-action ("CALL", "CLICK", "CLAIM") | Very frequent | Occasional |
| Personal pronouns (I, me, we) | Rare | Frequent |
| Numbers (phone, prize amounts) | Frequent | Occasional |

## 3.3 Visualisations Used

- Pie chart — spam vs. ham class distribution
- Histograms — character count, word count, sentence count per class
- Pairplot — cross-feature relationships by class
- Correlation heatmap — inter-feature correlations
- Word clouds — most frequent words in spam vs. ham separately

## 4. Text Preprocessing Pipeline

Raw SMS text is inconsistent and noisy. A six-step cleaning pipeline (`transform_message()`) was applied identically at training time and inference time to guarantee no train/serve skew.

| Step | Operation | Example |
|---|---|---|
| 1 | Lowercase conversion | "FREE PRIZE" → "free prize" |
| 2 | Tokenisation (NLTK) | "free prize" → ["free", "prize"] |
| 3 | Alphanumeric filter | Remove "!", "$", "@", etc. |
| 4 | Stopword removal (NLTK) | Remove "the", "is", "at", etc. |
| 5 | Punctuation removal | Remove remaining punctuation tokens |
| 6 | Porter Stemming | "claiming" → "claim", "prizes" → "prize" |

### Actual Implementation (from notebook):

```
def transform_message(text): text = text.lower() text = nltk.word_tokenize(text) words = [c for c in text if c.isalnum()] words = [w for w in words if w not in stopwords.words('english') and w not in string.punctuation] words = [ps.stem(w) for w in words] return " ".join(words)
```

**Why Porter Stemmer?** It aggressively reduces words to their roots, collapsing "winning", "winner", "wins" all to "win" — reducing vocabulary size and improving generalisation on unseen spam variations.

# 5. Feature Engineering — TF-IDF Vectorisation

The cleaned text was converted into a numerical matrix using **Term Frequency–Inverse Document Frequency (TF-IDF)** vectorisation.

## 5.1 Why TF-IDF over Bag of Words?

| Property | Bag of Words (CountVectorizer) | TF-IDF |
|---|---|---|
| Feature values | Raw word counts | Weighted scores |
| Common word penalty | None | Yes (IDF penalises) |
| Discriminative power | Lower | Higher |
| Chosen for this project | No | Yes |

TF-IDF gives high scores to words that are frequent in a specific message but rare across the entire corpus — making discriminative spam words like "prize", "claim", "winner" stand out strongly.

## 5.2 Vectoriser Configuration

| Parameter | Value | Reason |
|---|---|---|
| max_features | 3000 | Limits vocabulary to top 3000 words — reduces noise from rare words |
| ngram_range | Default (1,1) | Unigrams — sufficient for this domain |
| fit on | X2_train only | IDF weights computed on training set only — no data leakage |

**Critical note:** At inference time, only `vectorizer.transform()` is called — never `fit_transform()`. This ensures the exact same vocabulary and IDF weights are used as during training. The fitted vectoriser is saved as `vectorizer.pkl`.

# 6. Model Selection Journey

Multiple classifiers were evaluated systematically using 5-fold Stratified Cross-Validation before selecting the final ensemble.

## 6.1 Phase 1 — Naive Bayes Baseline

| Model | Accuracy | Precision | Notes |
|---|---|---|---|
| BernoulliNB | ~97% | ~98% | Performing well — good baseline |
| MultinomialNB | ~98% | ~100% | Best precision in this phase |
| GaussianNB | ~87% | ~71% | Worst performer — dropped |

MultinomialNB emerged as the strongest baseline — it is well-suited for TF-IDF weighted integer-like features and naturally handles the high-dimensional sparse text representation.

## 6.2 Phase 2 — Broader Model Evaluation

A wide range of classifiers was benchmarked using `evaluate_baselines_cv()` with 5-fold Stratified CV:

| Model | Strengths | Result |
|---|---|---|
| Logistic Regression | Linear, interpretable | Good accuracy, moderate precision |
| SVC (sigmoid kernel) | Strong margin-based boundary | High precision — selected |
| MultinomialNB | Fast, text-native | Best precision — selected |
| ExtraTreesClassifier | Robust ensemble tree | High precision — selected |
| RandomForestClassifier | Stable ensemble | Good but outperformed by ETC |
| KNeighborsClassifier | Instance-based | Weak on text features |
| GradientBoostingClassifier | Sequential boosting | Slow, marginal gain |
| AdaBoostClassifier | Boosted weak learners | Weaker on this dataset |
| XGBoost (tuned) | Optimised via Optuna | Strong but heavy |

## 6.3 Hyperparameter Optimisation (XGBoost)

XGBoost was tuned using **Optuna** with 30 trials of Bayesian optimisation, tuning: n_estimators, max_depth, learning_rate, subsample, colsample_bytree. Despite high absolute performance, its complexity and inference overhead made it less suitable than the lighter ensemble of SVC + MNB + ETC.

# 7. Final Model — Soft Voting Ensemble

The final model is a **Soft Voting Classifier** combining three complementary estimators, each bringing a different inductive bias.

## 7.1 Architecture

| Component | Type | Configuration | Role in Ensemble |
|---|---|---|---|
| SVC | Support Vector Machine | kernel='sigmoid', gamma=1.0, probability=True, random_state=2 | Strong margin-based boundary on high-dimensiona |
| MultinomialNB | Naive Bayes | Default parameters | Fast probabilistic baseline — excellent on TF-IDF t |
| ExtraTreesClassifier | Ensemble Trees | n_estimators=100, random_state=2, n_jobs=-1 | Captures non-linear feature interactions robustly |

## 7.2 Why Soft Voting?

In **soft voting**, each classifier outputs a probability for each class. These probabilities are averaged to produce the final probability. This is superior to hard voting (majority vote on labels) because:

- Confident predictions are weighted more heavily than uncertain ones
- Reduces variance — no single weak classifier can dominate the decision
- Enables a continuous probability output required for custom threshold tuning
- The SVC, MNB, and ETC each capture different signal — their average is more stable

## 7.3 Ensemble Code

```
voting_clf = VotingClassifier( estimators=[ ('svc', SVC(kernel='sigmoid', gamma=1.0,
probability=True, random_state=2)), ('mnb', MultinomialNB()), ('etc', ExtraTreesClassifier(
n_estimators=100, random_state=2, n_jobs=-1)), ], voting='soft' )
```

# 8. Custom Threshold Tuning

By default, classifiers use a 0.5 probability threshold to assign class labels. This project replaces that with a **custom precision-optimised threshold**.

## 8.1 Why Custom Threshold?

In this domain, a **false positive** (flagging a legitimate message as spam) is more harmful than a **false negative** (missing spam). Raising the spam probability threshold means the model only flags something as spam when it is extremely confident — maximising precision at a controlled recall trade-off.

## 8.2 How the Threshold is Found

```
def find_threshold_for_precision( clf, X_train, y_train, target_precision=0.999, n_splits=5): # Out-of-fold probabilities via StratifiedKFold CV oof_proba = cross_val_predict( clf, X_train, y_train, cv=cv, method='predict_proba')[:, 1] # Sweep precision-recall curve precisions, recalls, thresholds = \ precision_recall_curve(y_train, oof_proba) # Find lowest threshold meeting target precision for thr, prec, rec in zip(thresholds, precisions, recalls): if prec >= target_precision: return thr
```

**Important:** The threshold is found using **only training data** via out-of-fold cross-validation — the test set is never seen during this process, preventing any data leakage. The tuned threshold is saved to `threshold.json` and loaded by the API at runtime.

## 8.3 Target Precision

The target precision was set to **0.999** — meaning the model must be at least 99.9% confident before labelling a message as spam. This near-zero false-positive rate is essential for a trustworthy user-facing product.

## 9. Inference Pipeline

At prediction time, a new SMS message passes through an identical pipeline to what was used during training:

| Step | Component | File | Output |
|---|---|---|---|
| 1 | Text cleaning | utils/preprocess.py | Cleaned string |
| 2 | TF-IDF transform | model/vectorizer.pkl | Dense numpy array (shape: 1 × 3000) |
| 3 | Probability prediction | model/model.pkl | spam_probability ∈ [0, 1] |
| 4 | Threshold comparison | model/threshold.json | prediction: 0 (ham) or 1 (spam) |
| 5 | Safety score | utils/confidence.py | (1 – spam_prob) × 100 |
| 6 | Confidence label | utils/confidence.py | One of 20 band labels |

### 9.1 Critical: Sparse → Dense Conversion

The SVC component of the VotingClassifier was trained on **dense arrays** (`.toarray()` was called in the notebook). Since `vectorizer.transform()` returns a **sparse matrix** by default, the conversion must be applied explicitly at inference:

```
vector = vectorizer.transform([cleaned]).toarray() # sparse → dense
```

## 10. Confidence Score System

The raw spam probability is transformed into a **safety score** on a 0–100% scale and mapped to one of 20 human-readable confidence bands:

```
safety_score = (1 - spam_probability) * 100
```

| Range | Label | Range | Label |
|---|---|---|---|
| 0–5% | Extremely likely spam | 50–55% | Borderline, leaning safe |
| 5–10% | Very highly likely spam | 55–60% | Slightly leaning safe |
| 10–15% | Highly likely spam | 60–65% | Somewhat likely safe |
| 15–20% | Strongly likely spam | 65–70% | Probably safe |
| 20–25% | Likely spam | 70–75% | Likely safe |
| 25–30% | Probably spam | 75–80% | Strongly likely safe |
| 30–35% | Somewhat likely spam | 80–85% | Highly likely safe |
| 35–40% | Slightly leaning spam | 85–90% | Very highly likely safe |
| 40–45% | Borderline, leaning spam | 90–95% | Extremely likely safe |
| 45–50% | Borderline, still leaning spam | 95–100% | Highly likely not spam |

Every band from 0%–50% carries spam-leaning language. The spam/safe linguistic split occurs exactly at the 50% midpoint of the scale.

## 11. Serialised Model Artefacts

| File | Contents | Format | Critical Notes |
|---|---|---|---|
| model.pkl | Trained VotingClassifier (SVC + MultinomialNB + ExtraTreesClassifier) | pickle | Takes dense X2 array — must pass .toarray() at i |
| vectorizer.pkl | Fitted TfidfVectorizer (vocabulary + IDF weights, max_features=3000) | pickle | Only ever call .transform() at inference — never .fit_transform() |
| threshold.json | Custom decision threshold (float) | JSON | Found via out-of-fold CV on training data only |

**Version pinning:** All artefacts were serialised with scikit-learn **1.6.1**. The same version must be installed in every environment (local, Render) to avoid InconsistentVersionWarning or unpickling errors. This is enforced via `requirements.txt`.

## 12. Deployment Architecture

The application is deployed as a single **Flask** web service on **Render**.

| Layer | Technology | Details |
|---|---|---|
| Web Server (prod) | Gunicorn | WSGI server — declared in Procfile |
| API Framework | Flask 3.0.3 | Serves both API routes and static frontend files |
| CORS | flask-cors 4.0.1 | Allows cross-origin requests during development |
| Static Serving | Flask send_from_directory | Frontend HTML/CSS/JS served from /frontend |
| Model Loading | At startup | Artefacts loaded once — not on every request |
| Platform | Render | Detected via PORT env variable |

Running with a single `python app.py` command starts the Flask development server. In production, Gunicorn replaces this via the Procfile: `web: gunicorn app:app --chdir backend --bind 0.0.0.0:$PORT`